ALOUANI Fadi

BTS SIO 1
SLAM
16/10/26

TD2 : Les TRIGGERS

Sommaire

Introduction....1
Exercice 1 : parc immobilier....1
Exercice 2 : des produits au prix variant au fil du temps....2
Exercice 3 : location de véhicules....2
Conclusion....2

Introduction

Dans ce TP, j'ai découvert les triggers en SQL avec MySQL. J'ai appris à créer des déclencheurs pour vérifier des contraintes et automatiser certaines mises à jour dans une base de données.

ALOUANI Fadi

Exercice 1 : parc immobilier

Création de la DB :

```
exercice1.sql
exercice1.sql > ...
20      chauffage CHAR(1) NOT NULL,
21      placeParking TINYINT(1) NOT NULL,
22      prixParking DOUBLE,
23      PRIMARY KEY (immeuble, num),
24      FOREIGN KEY (immeuble) REFERENCES Immeuble(id)
25  );
26
27  CREATE TABLE Photo (
28      immeuble INT(11),
29      appartement INT(3),
30      reference INT(11) NOT NULL,
31      titre VARCHAR(75),
32      description VARCHAR(255),
33      uri VARCHAR(120) NOT NULL,
34      PRIMARY KEY (immeuble, appartement, reference),
35      FOREIGN KEY (immeuble, appartement)
36      REFERENCES Appartement(immeuble, num)
37  );
38
39  CREATE TABLE Piece (
40      immeuble INT(11),
41      appartement INT(3),
42      num INT(2) NOT NULL,
43      superficie DOUBLE,
44      fonction VARCHAR(30),
45      PRIMARY KEY (immeuble, appartement, num),
46      FOREIGN KEY (immeuble, appartement)
47      REFERENCES Appartement(immeuble, num)
48  );
49
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\eleve\TP5_SQL_Triggers> mysql -u root -p
mysql> cls
    -> ^C
mysql> clear
mysql> CREATE DATABASE IF NOT EXISTS tp5_triggers;
Query OK, 1 row affected (0.02 sec)

mysql> USE tp5_triggers;SOURCE C:/Users/eleve/TP5_SQL_Triggers/exercice1.sql;
Database changed
Query OK, 0 rows affected (0.02 sec)

Query OK, 0 rows affected (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

mysql>
```

```
mysql> SHOW TABLES;
+------------------------+
| Tables_in_tp5_triggers |
+------------------------+
| appartement            |
| immeuble               |
| photo                  |
| piece                  |
+------------------------+
4 rows in set (0.00 sec)
```

1). Rédiger le trigger permettant de vérifier la contrainte suivante : le prix de la place de parking d'un appartement peut et doit être NULL si l'appartement ne possède pas de place de parking. Tester le bon fonctionnement de votre trigger

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER verif_prix_parking
    -> BEFORE INSERT ON Appartement
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.placeParking = 0 THEN
    ->         SET NEW.prixParking = NULL;
    ->     END IF;
    -> END//
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

```
PS C:\Users\eleve\TP5_SQL_Triggers> mysql -u root -p
    -> SELECT * FROM Appartement;INSERT INTO Immeuble
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your Maria
rCodePostal, adrVille, fibreOptique, parkingPrivatif)
VALU...' at line 1
    -> (adrNum, adrVoie, adrCodePostal, adrVille, fibreOptique, parkingPrivatif)
    -> VALUES ('10', 'Rue de Paris', '05000', 'Gap', 1, 1);
    ->
    -> INSERT INTO Appartement
    -> VALUES (1, 1, 'Appartement test', 650, 50, 0, 'C', 'E', 0, 100);
    ->
    -> SELECT * FROM Appartement;
    -> ^C
mysql> DELIMITER ;
mysql> INSERT INTO Immeuble
    -> (adrNum, adrVoie, adrCodePostal, adrVille, fibreOptique, parkingPrivatif)
    -> VALUES ('10', 'Rue de Paris', '05000', 'Gap', 1, 1);
Query OK, 1 row affected (0.05 sec)

mysql> INSERT INTO Appartement
    -> VALUES (1, 1, 'Appartement test', 650, 50, 0, 'C', 'E', 0, 100);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT placeParking, prixParking
    -> FROM Appartement;
+--------------+-------------+
| placeParking | prixParking |
+--------------+-------------+
|            0 |        NULL |
+--------------+-------------+
1 row in set (0.00 sec)
```

J'ai créé un trigger BEFORE INSERT qui vérifie si l'appartement possède une place de parking. Si ce n'est pas le cas, le prix du parking est automatiquement mis à NULL. J'ai testé le trigger et le résultat est correct.

2)On souhaite que la contrainte suivante soit vérifiée : la superficie totale d'un appartement doit être égale à la somme de la superficie de chacune de ses pièces. Pour ce faire, créer le trigger qui permet de mettre à jour la superficie d'un appartement à l'insertion d'une pièce.

```
mysql> CREATE TRIGGER ajout_piece
    -> AFTER INSERT ON Piece
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE Appartement
    ->     SET superficie = (
    ->         SELECT SUM(superficie)
    ->         FROM Piece
    ->         WHERE immeuble = NEW.immeuble
    ->         AND appartement = NEW.appartement
    ->     )
    ->     WHERE immeuble = NEW.immeuble
    ->     AND num = NEW.appartement;
    -> END//
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Piece
    -> VALUES (1, 1, 1, 20, 'Chambre');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Piece
    -> VALUES (1, 1, 2, 30, 'Salon');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT superficie
    -> FROM Appartement
    -> WHERE immeuble = 1 AND num = 1;
+------------+
| superficie |
+------------+
|         50 |
+------------+
```

J'ai créé un trigger AFTER INSERT qui calcule automatiquement la superficie totale d'un appartement en additionnant les superficies de ses pièces. J'ai effectué un test avec une chambre de 20 m² et un salon de 30 m². Le résultat obtenu est bien de 50 m².

3)Adapter le trigger de la question 1 afin : - de vérifier la contrainte suivante : un appartement ne peut avoir de place de parking si l'immeuble n'a pas de parking privatif ; - d'initialiser la superficie de l'appartement à 0 à l'insertion d'un appartement

```
mysql> DROP TRIGGER verif_prix_parking;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER verif_prix_parking
    -> BEFORE INSERT ON Appartement
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE parking INT;
    ->
    ->     SELECT parkingPrivatif
    ->     INTO parking
    ->     FROM Immeuble
    ->     WHERE Id = NEW.immeuble;
    ->
    ->     IF NEW.placeParking = 0 THEN
    ->         SET NEW.prixParking = NULL;
    ->     ELSE
    ->         IF parking = 0 THEN
    ->             SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Cet immeuble ne possède pas de parking privatif';
    ->         END IF;
    ->     END IF;
    -> END//
Query OK, 0 rows affected (0.02 sec)
```

```
mysql>
mysql> DELIMITER ;
mysql>
mysql>
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER init_superficie
    -> BEFORE INSERT ON Appartement
    -> FOR EACH ROW
    -> BEGIN
    ->     SET NEW.superficie = 0;
    -> END//
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Immeuble
    -> (adrNum, adrVoie, adrCodePostal, adrVille, fibreOptique, parkingPrivatif)
    -> VALUES ('12', 'Rue des Alpes', '05000', 'Gap', 1, 0);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Appartement
    -> VALUES (2, 1, 'Appartement test 2', 700, 80, 0, 'B', 'E', 0, 100);
Query OK, 1 row affected (0.02 sec)

mysql> SELECT immeuble, num, superficie, placeParking, prixParking
    -> FROM Appartement;
+----------+-----+------------+--------------+-------------+
| immeuble | num | superficie | placeParking | prixParking |
+----------+-----+------------+--------------+-------------+
|        1 |   1 |         50 |            0 |        NULL |
|        2 |   1 |          0 |            0 |        NULL |
+----------+-----+------------+--------------+-------------+
```

4)En vous inspirant du trigger de la question 2, rédiger celui qui permet de mettre à jour la superficie d'un appartement à la mise à jour d'une pièce. Rédiger également le trigger qui met à jour la superficie d'un appartement à la suppression d'une pièce

```
mysql>
mysql> CREATE TRIGGER modifier_piece
    -> AFTER UPDATE ON Piece
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE Appartement
    ->     SET superficie = (
    ->         SELECT COALESCE(SUM(superficie), 0)
    ->         FROM Piece
    ->         WHERE immeuble = NEW.immeuble
    ->         AND appartement = NEW.appartement
    ->     )
    ->     WHERE immeuble = NEW.immeuble
    ->     AND num = NEW.appartement;
    -> END//
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER supprimer_piece
    -> AFTER DELETE ON Piece
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE Appartement
    ->     SET superficie = (
    ->         SELECT COALESCE(SUM(superficie), 0)
    ->         FROM Piece
    ->         WHERE immeuble = OLD.immeuble
    ->         AND appartement = OLD.appartement
    ->     )
    ->     WHERE immeuble = OLD.immeuble
    ->     AND num = OLD.appartement;
    -> END//
Query OK, 0 rows affected (0.02 sec)
```

```
mysql>
mysql> DELIMITER ;
mysql>  UPDATE Piece
    -> SET superficie = 25
    -> WHERE immeuble = 1
    -> AND appartement = 1
    -> AND num = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SELECT superficie
    -> FROM Appartement
    -> WHERE immeuble = 1 AND num = 1;
+------------+
| superficie |
+------------+
|         55 |
+------------+
1 row in set (0.00 sec)

mysql>
mysql> DELETE FROM Piece
    -> WHERE immeuble = 1
    -> AND appartement = 1
    -> AND num = 2;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT superficie
    -> FROM Appartement
    -> WHERE immeuble = 1 AND num = 1;
+------------+
| superficie |
+------------+
|         25 |
+------------+
1 row in set (0.00 sec)
```

ALOUANI Fadi

Exercice 2 : des produits au prix variant au fil du temps

1)Rédiger le script de création de table relatif à la base de données ci-avant décrite.

exercice1.sql | exercice2.sql

exercice2.sql > ...

```sql
CREATE TABLE Produit (
    code INT NOT NULL PRIMARY KEY,
    libelle VARCHAR(100) NOT NULL,
    prix DOUBLE NOT NULL,
    formation TINYINT(1) NOT NULL,
    poids DOUBLE NOT NULL
);

CREATE TABLE Mode_Livraison (
    num INT NOT NULL PRIMARY KEY,
    libelle VARCHAR(100) NOT NULL
);

CREATE TABLE Frais_Transport (
    livraison INT NOT NULL,
    num INT NOT NULL,
    prix DOUBLE NOT NULL,
    poidsMin DOUBLE NOT NULL,
    poidsMax DOUBLE NOT NULL,
    PRIMARY KEY (livraison, num),
    FOREIGN KEY (livraison) REFERENCES Mode_Livraison(num)
);

CREATE TABLE Commande (
    num INT NOT NULL PRIMARY KEY,
    destinataire VARCHAR(100) NOT NULL,
    prixTotal DOUBLE NOT NULL DEFAULT 0,
    livraison INT NULL,
    transport INT NULL,
    FOREIGN KEY (livraison, transport)
    REFERENCES Frais_Transport(livraison, num)
);
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\eleve\TP5_SQL_Triggers> mysql -u root -p

Query OK, 0 rows affected (0.03 sec)

Query OK, 0 rows affected (0.02 sec)

mysql> SHOW TABLES;
+------------------------+
| Tables_in_tp5_exercice2 |
+------------------------+
| commande               |
| frais_transport        |
| ligne_commande         |
| mode_livraison         |
| produit                |
+------------------------+
5 rows in set (0.00 sec)

mysql>
```

2)Rédiger le trigger permettant de vérifier la contrainte suivante : si une commande porte sur une formation, alors elle ne peut porter sur un ou plusieurs livres.

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER verif_formation
    -> BEFORE INSERT ON Ligne_Commande
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE typeProduit TINYINT;
    ->
    ->     SELECT formation INTO typeProduit
    ->     FROM Produit
    ->     WHERE code = NEW.produit;
    ->
    ->     IF typeProduit = 1 AND EXISTS (
    ->         SELECT *
    ->         FROM Ligne_Commande LC
    ->         INNER JOIN Produit P ON LC.produit = P.code
    ->         WHERE LC.commande = NEW.commande
    ->         AND P.formation = 0
    ->     ) THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Une commande ne peut pas contenir formations et livres';
    ->     END IF;
    -> END//
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

3)Rédiger le trigger permettant de vérifier la contrainte réciproque : si une commande porte sur un ou plusieurs livres, alors elle ne peut porter sur une formation.

```
mysql>
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER verif_livre
    -> BEFORE INSERT ON Ligne_Commande
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE typeProduit TINYINT;
    ->
    ->     SELECT formation INTO typeProduit
    ->     FROM Produit
    ->     WHERE code = NEW.produit;
    ->
    ->     IF typeProduit = 0 AND EXISTS (
    ->         SELECT *
    ->         FROM Ligne_Commande LC
    ->         INNER JOIN Produit P ON LC.produit = P.code
    ->         WHERE LC.commande = NEW.commande
    ->         AND P.formation = 1
    ->     ) THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Une commande ne peut pas contenir formations et livres';
    ->     END IF;
    -> END//
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

4)Adapter le trigger de la question 2 ou 3 afin de vérifier la contrainte suivante : si la commande porte sur une formation, alors il ne doit pas y avoir de frais de transport.

```
PS C:\Users\eleve\TP5_SQL_Triggers> mysql -u root -p
    ->     FROM Produit
    ->     WHERE code = NEW.produit;
    ->
    ->     IF typeProduit = 1 THEN
    ->
    ->         IF EXISTS (
    ->             SELECT *
    ->             FROM Ligne_Commande LC
    ->             INNER JOIN Produit P ON LC.produit = P.code
    ->             WHERE LC.commande = NEW.commande
    ->             AND P.formation = 0
    ->         ) THEN
    ->             SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Une commande ne peut pas contenir formations et liv
    ->         END IF;
    ->
    ->         UPDATE Commande
    ->         SET livraison = NULL,
    ->             transport = NULL
    ->         WHERE num = NEW.commande;
    ->
    ->     END IF;
    -> END//
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Mode_Livraison
    -> VALUES (1, 'Colissimo');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Frais_Transport
    -> VALUES (1, 1, 10, 0, 10);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Commande
    -> VALUES (3, 'Marie', 0, 1, 1);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Ligne_Commande
    -> VALUES (3, 1, 200, 1, 200);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT num, livraison, transport
    -> FROM Commande
    -> WHERE num = 3;
+-----+-----------+-----------+
| num | livraison | transport |
+-----+-----------+-----------+
|   3 |      NULL |      NULL |
+-----+-----------+-----------+
1 row in set (0.00 sec)
```

ALOUANI Fadi

J'ai adapté le trigger de la question 2 afin que lorsqu'une commande contient une formation, les informations de livraison et de transport soient automatiquement mises à NULL. J'ai effectué un test et le résultat est correct.

5)Adapter le trigger de la question 2 ou 3 afin que le prix total d'une commande soit automatiquement mis à jour à lorsqu'une ligne de commande est insérée.

```
    ->             AND P.formation = 0
    ->         ) THEN
    ->             SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Formation et livre incompatibles';
    ->         END IF;
    ->
    ->     ELSE
    ->
    ->         IF EXISTS (
    ->             SELECT *
    ->             FROM Ligne_Commande LC
    ->             INNER JOIN Produit P ON LC.produit = P.code
    ->             WHERE LC.commande = NEW.commande
    ->             AND P.formation = 1
    ->         ) THEN
    ->             SIGNAL SQLSTATE '45000'
    ->             SET MESSAGE_TEXT = 'Formation et livre incompatibles';
    ->         END IF;
    ->
    ->     END IF;
    ->
    ->     UPDATE Commande
    ->     SET prixTotal = prixTotal + NEW.prixt
    ->     WHERE num = NEW.commande;
    ->
    -> END//
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Commande (num, destinataire, prixTotal)
    -> VALUES (5, 'Lucas', 0);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Ligne_Commande
    -> VALUES (5, 2, 30, 2, 60);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Ligne_Commande
    -> VALUES (5, 3, 40, 1, 40);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
REIGN KEY (`produit`) REFERENCES `produit` (`code`))
mysql> SELECT num, prixTotal
    -> FROM Commande
    -> WHERE num = 5;
+-----+-----------+
| num | prixTotal |
+-----+-----------+
|   5 |        60 |
+-----+-----------+
1 row in set (0.00 sec)
```

6.) Préciser les cas que nous n'avons pas traités au travers des questions 1 à 5.

Plusieurs cas n'ont pas été traités dans les questions précédentes :

- La modification d'une ligne de commande (`UPDATE`).
- La suppression d'une ligne de commande (`DELETE`).
- La mise à jour automatique du prix total après une modification ou une suppression.
- La modification du prix d'un produit déjà présent dans une commande.
- La vérification des quantités et des prix négatifs.
- La vérification qu'une commande contenant une formation ne possède pas de frais de transport lors de sa modification.

Exercice 3 : location de véhicules

1)Créer sous MySQL une base de données correspondant au modèle de données ci-dessus (fournir le script de création de tables).

exercice1.sql exercice2.sql exercice3.sql

exercice3.sql > ...

```sql
CREATE TABLE Conducteur (
    id INT AUTO_INCREMENT PRIMARY KEY,
    civilite VARCHAR(20),
    prenom VARCHAR(50) NOT NULL,
    nom VARCHAR(50) NOT NULL
);

CREATE TABLE Permis (
    id INT AUTO_INCREMENT PRIMARY KEY,
    type VARCHAR(1) NOT NULL
);

CREATE TABLE Vehicule (
    id INT AUTO_INCREMENT PRIMARY KEY,
    immatriculation VARCHAR(20) NOT NULL,
    type VARCHAR(50) NOT NULL
);

CREATE TABLE Titulaire (
    conducteur INT NOT NULL,
    permis INT NOT NULL,
    PRIMARY KEY (conducteur, permis),
    FOREIGN KEY (conducteur) REFERENCES Conducteur(id),
    FOREIGN KEY (permis) REFERENCES Permis(id)
);
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\eleve\TP5_SQL_Triggers> mysql -u root -p
Query OK, 0 rows affected (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

Query OK, 0 rows affected (0.03 sec)

Query OK, 0 rows affected (0.02 sec)

Query OK, 0 rows affected (0.03 sec)

mysql>  SHOW TABLES;
+-------------------------+
| Tables_in_tp5_exercice3 |
+-------------------------+
| conducteur              |
| location                |
| necessiter              |
| permis                  |
| titulaire               |
| vehicule                |
+-------------------------+
6 rows in set (0.00 sec)

mysql>
```

2.) Rédiger le trigger qui permet, à l'insertion et à la modification d'une Location (INSERT ou UPDATE sur la table « Location »), de vérifier que le conducteur spécifié correspond à un conducteur ayant parmi ses permis celui requis pour conduire le véhicule.

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER verif_permis_insert
    -> BEFORE INSERT ON Location
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NOT EXISTS (
    ->         SELECT *
    ->         FROM Titulaire T
    ->         INNER JOIN Necessiter N ON T.permis = N.permis
    ->         WHERE T.conducteur = NEW.conducteur
    ->         AND N.vehicule = NEW.vehicule
    ->     ) THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Le conducteur ne possède pas le permis nécessaire';
    ->     END IF;
    -> END//
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER verif_permis_update
    -> BEFORE UPDATE ON Location
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NOT EXISTS (
    ->         SELECT *
    ->         FROM Titulaire T
    ->         INNER JOIN Necessiter N ON T.permis = N.permis
    ->         WHERE T.conducteur = NEW.conducteur
    ->         AND N.vehicule = NEW.vehicule
    ->     ) THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Le conducteur ne possède pas le permis nécessaire';
    ->     END IF;
    -> END//
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO Conducteur (civilite, prenom, nom)
    -> VALUES
    -> ('M.', 'Jean', 'Dupont'),
    -> ('M.', 'Paul', 'Martin');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Permis (type)
    -> VALUES ('B');
Query OK, 1 row affected (0.00 sec)
```

```
mysql>
mysql> INSERT INTO Permis (type)
    -> VALUES ('B');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO Titulaire
    -> VALUES (1, 1);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO Vehicule (immatriculation, type)
    -> VALUES ('AB-123-CD', 'Voiture');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO Necessiter
    -> VALUES (1, 1);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Location (conducteur, vehicule, debut, fin)
    -> VALUES (1, 1, '2026-11-14 10:00:00', '2026-11-15 10:00:00');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Location (conducteur, vehicule, debut, fin)
    -> VALUES (2, 1, '2026-11-16 10:00:00', '2026-11-17 10:00:00');
ERROR 1644 (45000): Le conducteur ne possède pas le permis nécessaire
mysql> UPDATE Location
    -> SET conducteur = 2
    -> WHERE id = 1;
ERROR 1644 (45000): Le conducteur ne possède pas le permis nécessaire
mysql>
```

ALOUANI Fadi

J'ai créé deux triggers, un pour l'insertion et un pour la modification d'une location. Ils vérifient que le conducteur possède bien le permis nécessaire pour conduire le véhicule. J'ai effectué plusieurs tests et les locations sans permis adapté sont correctement refusées.

Conclusion

Ce TP m’a permis de mieux comprendre le fonctionnement des triggers et leur utilité. J’ai appris à automatiser des actions et à contrôler les données pour garder une base de données cohérente.